

# Cyclistic Bike Share: Capstone Case Study

## Introduction

This case study is the capstone project for the Google Data Analytics Professional Certificate. It examines how annual members and casual riders use Cyclistic's bike-share service differently, following the six-step data analysis process: **Ask, Prepare, Process, Analyze, Share, and Act**. SQL was used for analysis and Tableau for visualization.

## Background

### Cyclistic

Cyclistic is a bike-share program that features more than 5,800 bicycles and 600 docking stations. Cyclistic sets itself apart by also offering reclining bikes, hand tricycles, and cargo bikes, making bike share more inclusive to people with disabilities and riders who cannot use a standard two-wheeled bike. Most riders opt for traditional bikes; about 8% of riders use the assistive options. Users are more likely to ride for leisure, but about 30% use them to commute to work each day.

The marketing strategy relied on building general awareness and appealing to broad consumer segments. One approach that helped make these things possible was the flexibility of its pricing plans: single-ride passes, full-day passes, and annual memberships. Customers who purchase single-ride or full-day passes are referred to as casual riders. Customers who purchase annual memberships are Cyclistic members.

The finance analysts have concluded that annual members are much more profitable than casual riders. Although the pricing flexibility helps Cyclistic attract more customers, Lily Moreno (the director of marketing and my manager) believes that maximizing the number of annual members will be key to future growth. Rather than creating a marketing campaign that targets all-new customers, Lily Moreno believes there is a good chance to convert casual riders into members. She notes that casual riders are already aware of the Cyclistic program and have chosen Cyclistic for their mobility needs.

### Scenario

I am assuming the role of a junior data analyst in the marketing team at Cyclistic bike-share company. The director of marketing believes that company's future success depends on maximizing the number of annual memberships. My team would like to understand how casual and annual members use Cyclistic bikes differently. From the insights, my team will design a marketing strategy to convert casual riders into annual membership. But first, Cyclistic executives must approve my recommendations, so they must be backed by compelling data insights and professional data visualizations.

## Ask

**Business Task: Converting casual riders into membership riders:**

## Analysis Questions:

Three questions will guide the future marketing program:

1. How do annual members and casual riders use Cyclistic bikes differently?
2. Why would casual riders buy Cyclistic annual memberships?
3. How can Cyclistic use digital media to influence casual riders to become members?

The director of marketing and my manager *Lily Moreno* has assigned me the first question to answer: *How do annual members and casual riders use Cyclistic bikes differently?*

## Key Tasks

**Identify business tasks:**

- The main objective is to build the best marketing strategies to turn casual bike riders into annual members by analyzing how the 'Casual' and 'Annual' customers use Cyclistic bike share differently.

**Identify key stakeholders:**

- Cyclistic executive team, Director of Marketing (Lily Moreno), Marketing Analytics team.

## Deliverable

**A clear statement of the business task:**

- Find the differences between casual and member riders.

## Prepare

### Data source

I utilized Cyclistic's historical trip data from Apr 2025 to Mar 2026. This is a public dataset published by Motivate International Inc under this license. I selected this period because it is more recent data. As the data source files are large, I used SQL Server to combine the twelve .csv files into one file consisting of 5.6 million rows.

This is public data that can be used to explore how different customer types are using Cyclistic bikes. But note that data-privacy issues prohibit riders from using personal identifiable information. This means that we will not be able to connect pass purchases to credit card numbers to determine if casual riders live in the Cyclistic service area or if they have purchased multiple single passes.

## Data Organization

The naming convention for the files of YYYYMM-divvy-tripdata, and each file includes information for one month such as the ride id, bike type, start time, end time, start station, end station, start location, end location, and whether the rider is a member or not. The files contain thirteen identical attributes on each. The corresponding column names are ride\_id, rideable\_type, started\_at, ended\_at, start\_station\_name, start\_station\_id, end\_station\_name, end\_station\_id, start\_lat, start\_lng, end\_lat, end\_lng and member\_casual.

## Process

Utilized SQL Server Management 19 to combine the twelve tables into one dataset. In addition, I used SQL Server to clean the consolidated dataset. I used SQL Server because of the size limitation of Microsoft Excel since these files were large.

## Combining the data - [View Script](#)

12 csv files uploaded into SQL Server tables created in database 'Capstone\_TripData' from table names dbo.tripdata\_divvy\_YYYYMM. Another table named dbo.divvy\_tripdata\_consolidated created to combine the 12 SQL tables resulting in 5,620,544 combined rows of data for the year.

### -- Step 1: Create table to combine the twelve-monthly SQL tables

```
IF NOT EXISTS (
    SELECT *
    FROM sys.tables
    WHERE name = 'divvy_tripdata_consolidated' AND schema_id = SCHEMA_ID('dbo')
)
BEGIN
    CREATE TABLE dbo.[divvy_tripdata_consolidated] (
        ride_id NVARCHAR(50),
        rideable_type NVARCHAR(20),
        started_at DATETIME2(7),
        ended_at DATETIME2(7),
        start_station_name NVARCHAR(70),
        start_station_id NVARCHAR(40),
        end_station_name NVARCHAR(70),
        end_station_id NVARCHAR(40),
        start_lat FLOAT,
        start_lng FLOAT,
        end_lat FLOAT,
        end_lng FLOAT,
        member_casual NVARCHAR(10)
    );
END;
```

### -- Step 2: Populate the combine SQL table from twelve-monthly SQL tables

```
INSERT INTO dbo.[divvy_tripdata_consolidated] (
    ride_id,
    rideable_type,
    started_at,
    ended_at,
    start_station_name,
    start_station_id,
    end_station_name,
```

```

        end_station_id,
        start_lat,
        start_lng,
        end_lat,
        end_lng,
        member_casual
    )
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202504]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202505]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202506]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202507]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202508]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202509]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202510]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202511]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202512]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202601]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202602]
    UNION ALL
    SELECT * FROM [Capstone_Tripdata].[dbo].[tripdata_divvy_202603];

```

Then checked total row numbers, by utilizing SQL query below. The new dataset "**divvy\_tripdata\_consolidated**" contains a total of 5,620,544 data rows for the 12 months:

```

SELECT Count(*)
FROM [Capstone_Tripdata].[dbo].[divvy_tripdata_consolidated]

```

I used the script below to spot check the consolidated data by reviewing the first ten rows of the dataset.

```

SELECT TOP 10 *
FROM dbo.[divvy_tripdata_consolidated];

```

## Data Exploration - View Script

Before data cleaning, I took a closer look to ensure I understood the data and check for inconsistencies.

Discovered:

1. The column names are below with their data types.

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
AND TABLE_NAME = [divvy_tripdata_consolidated];
```

| Row | COLUMN_NAME        | DATA_TYPE |
|-----|--------------------|-----------|
| 1   | ride_id            | nvarchar  |
| 2   | rideable_type      | nvarchar  |
| 3   | started_at         | nvarchar  |
| 4   | ended_at           | nvarchar  |
| 5   | start_station_id   | nvarchar  |
| 6   | start_station_name | nvarchar  |
| 7   | end_station_id     | nvarchar  |
| 8   | end_station_name   | nvarchar  |
| 9   | start_lat          | float     |
| 10  | start_lng          | float     |
| 11  | end_lat            | float     |
| 12  | end_lng            | float     |
| 13  | member_casual      | nvarchar  |

2. The script below used to identify where **null values** exist. The line below script shows the columns with count of missing values.

```
SELECT
COUNT(*) - COUNT(ride_id) AS ride_id,
COUNT(*) - COUNT(rideable_type) AS rideable_type,
COUNT(*) - COUNT(started_at) AS started_at,
COUNT(*) - COUNT(ended_at) AS ended_at,
COUNT(*) - COUNT(CONVERT(nvarchar(max), start_station_name)) AS
start_station_name,
COUNT(*) - COUNT(CONVERT(nvarchar(max), start_station_id)) AS
start_station_id,
COUNT(*) - COUNT(CONVERT(nvarchar(max), end_station_name)) AS
end_station_name,
COUNT(*) - COUNT(CONVERT(nvarchar(max), end_station_id)) AS end_station_id,
COUNT(*) - COUNT(start_lat) AS start_lat,
COUNT(*) - COUNT(start_lng) AS start_lng,
COUNT(*) - COUNT(end_lat) AS end_lat,
COUNT(*) - COUNT(end_lng) AS end_lng,
COUNT(*) - COUNT(member_casual) AS member_casual
FROM dbo.[divvy_tripdata_consolidated]
```

| start_station_name | start_station_id | end_station_name | end_station_id | end_lat | end_lng |
|--------------------|------------------|------------------|----------------|---------|---------|
| 1,194,951          | 1,194,951        | 259,213          | 259,213        | 5,784   | 84      |

3. I ran script below and did not find any duplicates values.

```
SELECT COUNT(ride_id) - COUNT(DISTINCT ride_id) AS duplicate_rows
FROM dbo.[divvy_tripdata_consolidated];
```

4. Retrieve the records of the rideable\_type column to see the different bike types: electric\_bike, classic\_bike. I found rows for this column with leading and trailing quotation marks.

```
SELECT rideable_type, COUNT(rideable_type) AS trip_type
FROM dbo.[divvy_tripdata_consolidated]
GROUP BY rideable_type;
```

| Row | rideable_type   | trip_type |
|-----|-----------------|-----------|
| 1   | "electric_bike" | 3,678,925 |
| 2   | "classic_bike"  | 1,941,619 |

5. Retrieved records for member\_casual column to check the different member types: member, casual. I found rows for this column with leading and trailing quotation marks.

```
SELECT DISTINCT member_casual, COUNT(*) AS count_member_type
FROM dbo.[divvy_tripdata_consolidated]
GROUP BY member_casual;
```

| member_casual | count_member_type |
|---------------|-------------------|
| "member"      | 3,605,045         |
| "casual"      | 2,015,499         |

6. Rows with Null Values to be removed from dataset:

- In dataset, 1,194,952 rows where **start\_station\_name** and **start\_station\_id** values are missing.

```
SELECT DISTINCT
    CONVERT(nvarchar(max), start_station_name) AS start_station_name
FROM dbo.[divvy_tripdata_consolidated]
ORDER BY start_station_name;
```

```
SELECT COUNT(ride_id) AS start_station_null
FROM dbo.[divvy_tripdata_consolidated]
WHERE start_station_name IS NULL OR start_station_id IS NULL;
```

- In dataset, 1,259,214 rows where **end\_station\_name** and **end\_station\_id** values are missing.

```
SELECT COUNT(ride_id) AS end_station_null
FROM dbo.[divvy_tripdata_consolidated]
WHERE end_station_name IS NULL OR end_station_id IS NULL;
```

- In the dataset, 5,784 rows "**end\_lat**" and "**end\_lng**" values are missing.

```
SELECT COUNT(ride_id) AS end_loc_null
FROM dbo.[divvy_tripdata_consolidated]
WHERE end_lat IS NULL OR end_lng IS NULL;
```

## Planning for Data Cleaning

Trip start and end times are recorded in `started_at` and `ended_at` (format: YYYY-MM-DD hh:mm:ss UTC). Based on this, three additional columns were identified as needed for deeper analysis, to be added during the Data Cleaning step:

- **ride\_length** — total trip duration, calculated from `started_at` and `ended_at`
- **day\_of\_week** — the day of the week each trip started.
- **month** — the calendar month each trip started, to support seasonal trend analysis.

Two duration checks were also run during exploration to gauge how many rides would be affected by outlier filtering in the cleaning step:

- Rides under 1 minute: 151,875 rows
- Rides longer than 1 day: 5,808 rows

**Result:** These findings — missing values across station and location fields, the quotation-mark formatting issue, and the duration outliers — directly informed the exclusion criteria applied in the Data Cleaning step that follows.

- Columns such as "**day\_of\_week**" and "**month**" to represent the month, are added. "**month**" can offer valuable insights into analyzing trips at various times throughout the year.

```
SELECT TOP 20 started_at, ended_at
FROM dbo.[divvy_tripdata_consolidated];
```

```
SELECT COUNT(*) AS less_than_a_minute
FROM dbo.[divvy_tripdata_consolidated]
WHERE DATEDIFF(SECOND, started_at, ended_at) <= 60;  -- 60 seconds = 1
minute 139873 rows
```

```
SELECT COUNT(*) AS longer_than_1_day
FROM dbo.[divvy_tripdata_consolidated]
WHERE DATEDIFF(MINUTE, started_at, ended_at) >= 1440;  -- 1440 minutes = 1
day 7343 rows
```

## Data Cleaning - [View Script](#)

In this step, a new table (`divvy_tripdata_cleaned`) was created from the raw trip data used during exploration. The following steps were applied:

## 1. Removed rows with missing values

Rows missing values in start\_station\_name, end\_station\_name, started\_at, ended\_at, end\_lat, or end\_lng were excluded to preserve data integrity for analysis. A total of 1,945,725 rows were removed due to null values across these columns (individually or in combination).

## 2. Added three new calculated columns

- **ride\_length** — trip duration in minutes, calculated as the difference between ended\_at and started\_at
- **day\_of\_week** — the day of the week the ride started (Sun-Sat)
- **month** — the calendar month the ride started (Jan-Dec)

## 3. Excluded duration outliers

Rides shorter than 1 minute or longer than 24 hours (1,440 minutes) were removed, as these reflect false starts, docking errors, or maintenance/test rides rather than genuine trips.

## 4. Removed unused columns

start\_station\_id and end\_station\_id were excluded from the cleaned table since they were not needed for analysis or reporting.

## 5. Data type adjustments

ride\_length was set to INT (whole-minute values only), day\_of\_week/month to CHAR(3), and station name fields to NVARCHAR(70) — confirmed sufficient after checking the longest station name (64 characters).

## Result

**After cleaning, the table contains 3,674,812 rows, down from the original consolidated dataset.**

## SQL Script

### -- Create the new table

```
CREATE TABLE dbo.divvy_tripdata_cleaned(  
    ride_id nvarchar(50),  
    rideable_type nvarchar(20),  
    started_at datetime2(7),  
    ended_at datetime2(7),  
    ride_length INT,  
    day_of_week char(3),  
    month char(3),  
    start_station_name nvarchar(70),  
    end_station_name nvarchar(70),  
    start_lat float,  
    start_lng float,  
    end_lat float,  
    end_lng float,  
    member_casual nvarchar(10)  
);
```

### -- Insert data into the new table

```
WITH cleaned AS (  
    SELECT  
        REPLACE(a.ride_id, '', '') AS ride_id,
```



```

        REPLACE(a.rideable_type, '', '') AS rideable_type,
        TRY_CONVERT(datetime2, REPLACE(a.started_at, '', ''), 120) AS
started_at,
        TRY_CONVERT(datetime2, REPLACE(a.ended_at, '', ''), 120) AS ended_at,
        REPLACE(a.start_station_name, '', '') AS start_station_name,
        REPLACE(a.end_station_name, '', '') AS end_station_name,
        a.start_lat, a.start_lng, a.end_lat, a.end_lng,
        REPLACE(a.member_casual, '', '') AS member_casual
    FROM divvy_tripdata_consolidated a
)
INSERT INTO dbo.[divvy_tripdata_cleaned] (
    ride_id, rideable_type, started_at, ended_at,
    ride_length, day_of_week, month,
    start_station_name, end_station_name,
    start_lat, start_lng, end_lat, end_lng, member_casual)
SELECT
    ride_id,
    rideable_type,
    started_at,
    ended_at,
    DATEDIFF(MINUTE, started_at, ended_at) AS ride_length,
    CASE DATEPART(WEEKDAY, started_at)
        WHEN 1 THEN 'Sun' WHEN 2 THEN 'Mon' WHEN 3 THEN 'Tue'
        WHEN 4 THEN 'Wed' WHEN 5 THEN 'Thu' WHEN 6 THEN 'Fri'
        WHEN 7 THEN 'Sat'
    END AS day_of_week,
    CASE DATEPART(MONTH, started_at)
        WHEN 1 THEN 'Jan' WHEN 2 THEN 'Feb' WHEN 3 THEN 'Mar'
        WHEN 4 THEN 'Apr' WHEN 5 THEN 'May' WHEN 6 THEN 'Jun'
        WHEN 7 THEN 'Jul' WHEN 8 THEN 'Aug' WHEN 9 THEN 'Sep'
        WHEN 10 THEN 'Oct' WHEN 11 THEN 'Nov' WHEN 12 THEN 'Dec'
    END AS month,
    start_station_name,
    end_station_name,
    start_lat, start_lng, end_lat, end_lng,
    member_casual
FROM cleaned
WHERE
    start_station_name IS NOT NULL AND
    end_station_name IS NOT NULL AND
    started_at IS NOT NULL AND
    ended_at IS NOT NULL AND
    end_lat IS NOT NULL AND
    end_lng IS NOT NULL AND
    DATEDIFF(MINUTE, started_at, ended_at) > 1 AND
    DATEDIFF(MINUTE, started_at, ended_at) < 1440;

```

**-- Set ride\_id as the primary key for the new table**

```

ALTER TABLE divvy_tripdata_cleaned
ADD PRIMARY KEY (ride_id);

```

**-- Verify final row count**

```

SELECT COUNT(ride_id) AS no_of_rows
FROM dbo.[divvy_tripdata_cleaned];
-- Result: 3,674,812 rows

```

## Analyzed - View Script

The data prepared and ready for analysis to answer question: How do annual members and casual riders use Cyclistic bikes differently? To do this, ran calculations on a file to get a better sense of the data.

```
-- summary stats: mean/max ride_length, mode of day_of_week
```

```
SELECT
  AVG(ride_length) AS mean_ride_length,
  MAX(ride_length) AS max_ride_length
FROM dbo.[divvy_tripdata_cleaned];

SELECT TOP 1 day_of_week, COUNT(*) AS trip_count
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY day_of_week
ORDER BY trip_count DESC;
```

```
-- member vs. casual rides ride count comparison (KPI)
```

- KPI created directly in Tableau

```
-- bike type used by riders
```

```
SELECT member_casual, rideable_type, COUNT(*) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY member_casual, rideable_type
ORDER BY member_casual, total_trips;
```

```
-- number of trips per month (chronological order)
```

```
SELECT month, member_casual, COUNT(ride_id) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY month, member_casual
ORDER BY
  member_casual,
  CASE month
    WHEN 'Jan' THEN 1 WHEN 'Feb' THEN 2 WHEN 'Mar' THEN 3
    WHEN 'Apr' THEN 4 WHEN 'May' THEN 5 WHEN 'Jun' THEN 6
    WHEN 'Jul' THEN 7 WHEN 'Aug' THEN 8 WHEN 'Sep' THEN 9
    WHEN 'Oct' THEN 10 WHEN 'Nov' THEN 11 WHEN 'Dec' THEN 12
  END;
```

```
-- number of trips per day of week (chronological order, Sun-Sat)
```

```
SELECT day_of_week, member_casual, COUNT(ride_id) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY day_of_week, member_casual
ORDER BY
  member_casual,
  CASE day_of_week
    WHEN 'Sun' THEN 1 WHEN 'Mon' THEN 2 WHEN 'Tue' THEN 3
    WHEN 'Wed' THEN 4 WHEN 'Thu' THEN 5 WHEN 'Fri' THEN 6
    WHEN 'Sat' THEN 7
  END;
```

-- number of trips per hour (chronological order, 12 AM-11 PM)

```
SELECT
    FORMAT(started_at, 'hh tt') AS hour_of_day_am_pm,
    member_casual,
    COUNT(ride_id) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY
    FORMAT(started_at, 'hh tt'),
    member_casual
ORDER BY
    member_casual,
    CASE
        WHEN FORMAT(started_at, 'tt') = 'AM' THEN 0 ELSE 1
    END,
    CAST(FORMAT(started_at, 'hh') AS INT) % 12;
```

-- average ride\_length per month (chronological order)

```
SELECT month, member_casual, AVG(ride_length) AS avg_ride_duration
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY month, member_casual
ORDER BY
    member_casual,
    CASE month
        WHEN 'Jan' THEN 1 WHEN 'Feb' THEN 2 WHEN 'Mar' THEN 3
        WHEN 'Apr' THEN 4 WHEN 'May' THEN 5 WHEN 'Jun' THEN 6
        WHEN 'Jul' THEN 7 WHEN 'Aug' THEN 8 WHEN 'Sep' THEN 9
        WHEN 'Oct' THEN 10 WHEN 'Nov' THEN 11 WHEN 'Dec' THEN 12
    END;
```

-- average ride\_length per day of week (chronological order, Sun-Sat)

```
SELECT day_of_week, member_casual, AVG(ride_length) AS avg_ride_duration
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY day_of_week, member_casual
ORDER BY
    member_casual,
    CASE day_of_week
        WHEN 'Sun' THEN 1 WHEN 'Mon' THEN 2 WHEN 'Tue' THEN 3
        WHEN 'Wed' THEN 4 WHEN 'Thu' THEN 5 WHEN 'Fri' THEN 6
        WHEN 'Sat' THEN 7
    END;
```

-- average ride\_length per hour (chronological order, 12 AM-11 PM)

```
SELECT
    FORMAT(started_at, 'hh tt') AS hour_of_day_am_pm,
    member_casual,
    AVG(ride_length) AS avg_ride_duration
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY
    FORMAT(started_at, 'hh tt'),
    member_casual
ORDER BY
    member_casual,
    CASE
        WHEN FORMAT(started_at, 'tt') = 'AM' THEN 0 ELSE 1
    END,
    CAST(FORMAT(started_at, 'hh') AS INT) % 12;
```

### -- starting station locations

```
SELECT
    start_station_name,
    member_casual,
    AVG(start_lat) AS avg_start_lat,
    AVG(start_lng) AS avg_start_lng,
    COUNT(ride_id) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY
    start_station_name,
    member_casual;
```

### -- ending station locations

```
SELECT
    end_station_name,
    member_casual,
    AVG(end_lat) AS avg_end_lat,
    AVG(end_lng) AS avg_end_lng,
    COUNT(ride_id) AS total_trips
FROM dbo.[divvy_tripdata_cleaned]
GROUP BY
    end_station_name,
    member_casual;
```

## Share - [View Dashboard](#)

How do annual members and casual riders use Cyclistic bikes differently?

### KPI: Total Trips by Member vs. Casual Riders

---

2,364,113 1,310,699

---

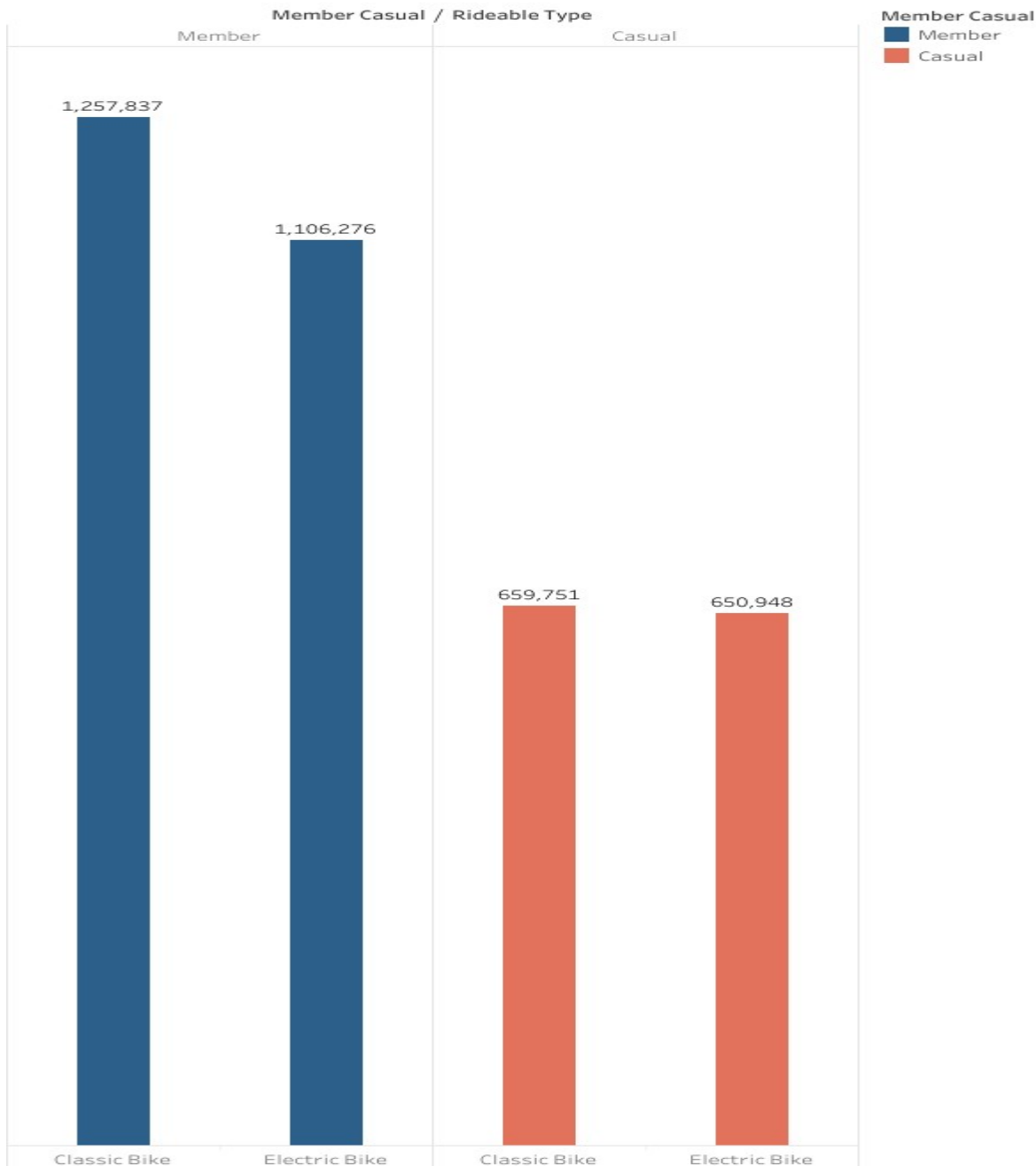
Member Casual

■ Member

■ Casual

1. Bike Types used by members vs. casual riders.

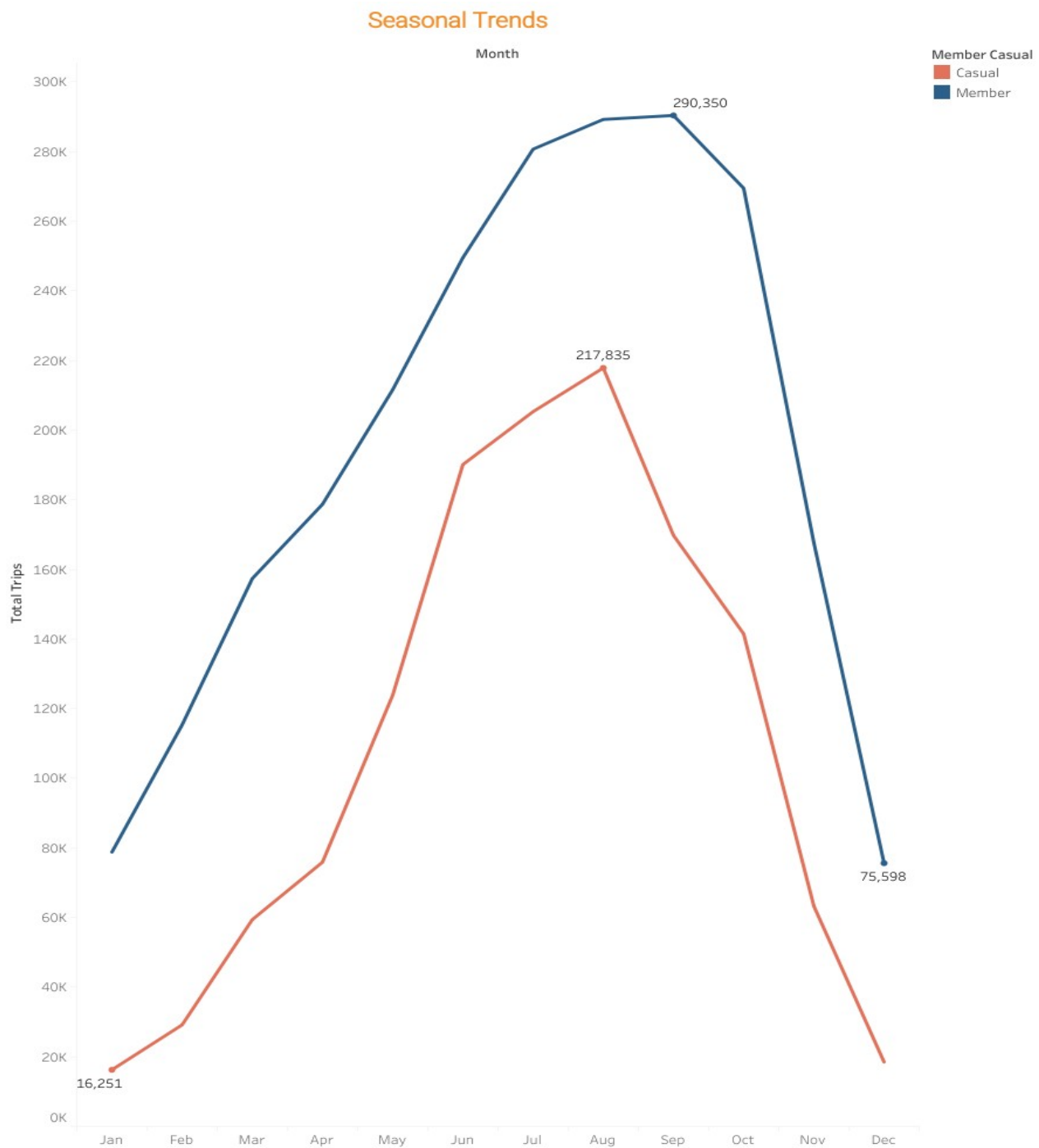
## Bike Type Usage: Member vs. Casual



The members make up 64.33% of total trip while casual riders took remaining 35.67%. The classic bike accounted for 52.18 percent of usage and electric bikes used 47.82. Both members and casual riders used slightly more classic bikes than electric bikes. But casual riders do not have a clear preference in bike type.

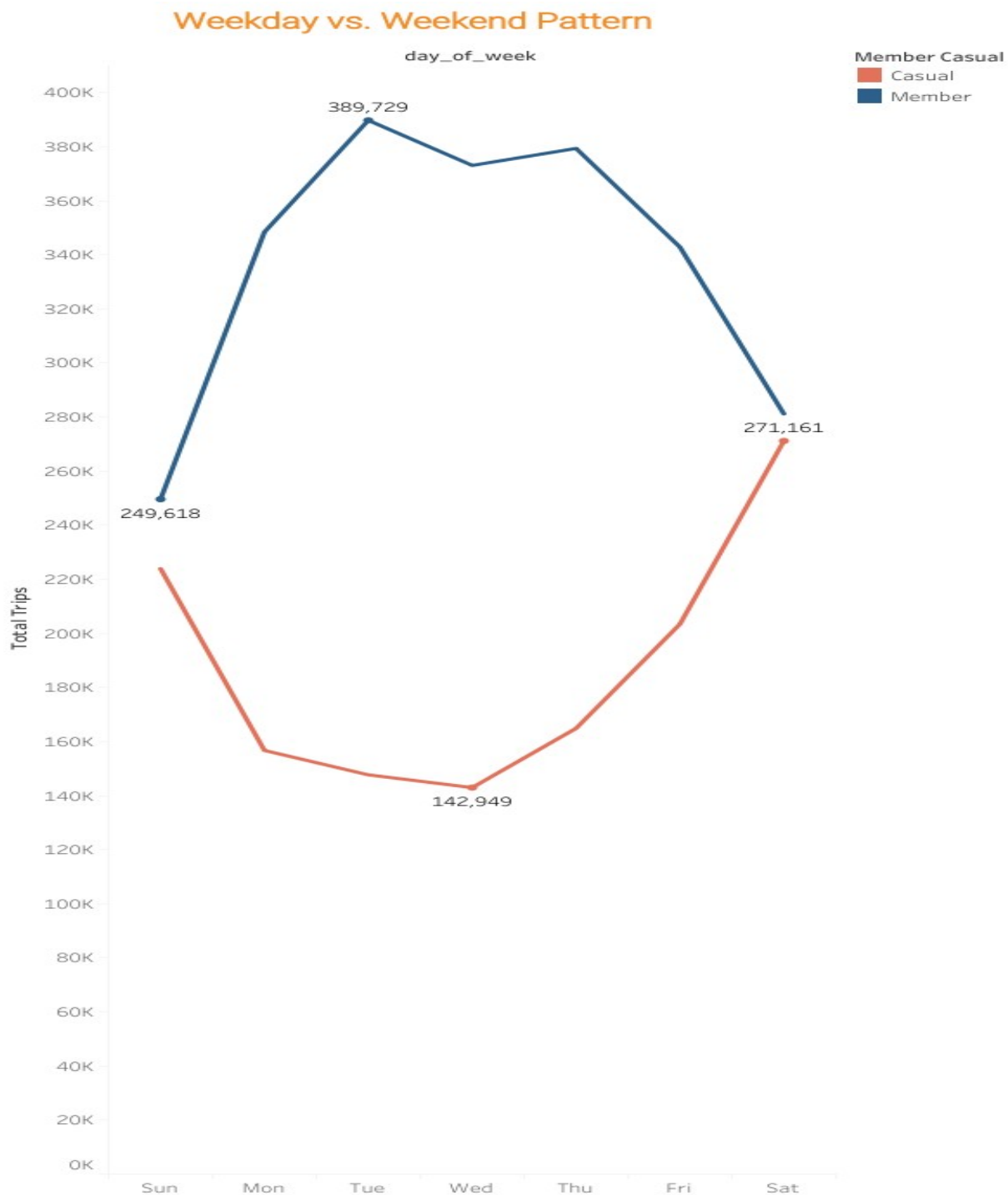
Below charts show Temporal & Usage Pattern Analysis. These charts help to examine member usage versus leisure and sightseeing riders for casual riders. They cover Number of Rides by month, day of the week & hour of day from April 2025 to March 2026.

2. **Month (Seasonal Trends):** Analyzed monthly trips to measure seasonal sensitivity and weather impact on members and casual riders.



Casual riders and members exhibit comparable behavior, with more trips in the spring and summer with fewer in the winter. The ridership for these two groups peaks between August and September and drops to the lowest from December to January. The gap between casuals and members is closest during the month of June because of vacation and increase in leisure ridership.

3. **Days of Week (Commuter vs. Leisure):** Compared total rides by user type for weekdays versus weekends which shows regular commuter versus leisure trends.



For days of the week, members take more trips during the week and their ridership declines during the weekend. Casual riders take fewer trips during the week and more on weekends when members' ridership declines. The gap between casual riders and members is closest on weekends since member ridership declined since they are off from work.

- 4. Hours of the Day (Peak Analysis):** Hourly ride counts (24-hour clock) by user type. This demonstrates classic commuting during peaks (7-9 AM and 4-6 PM) versus afternoon leisure humps (11 AM-5 PM).

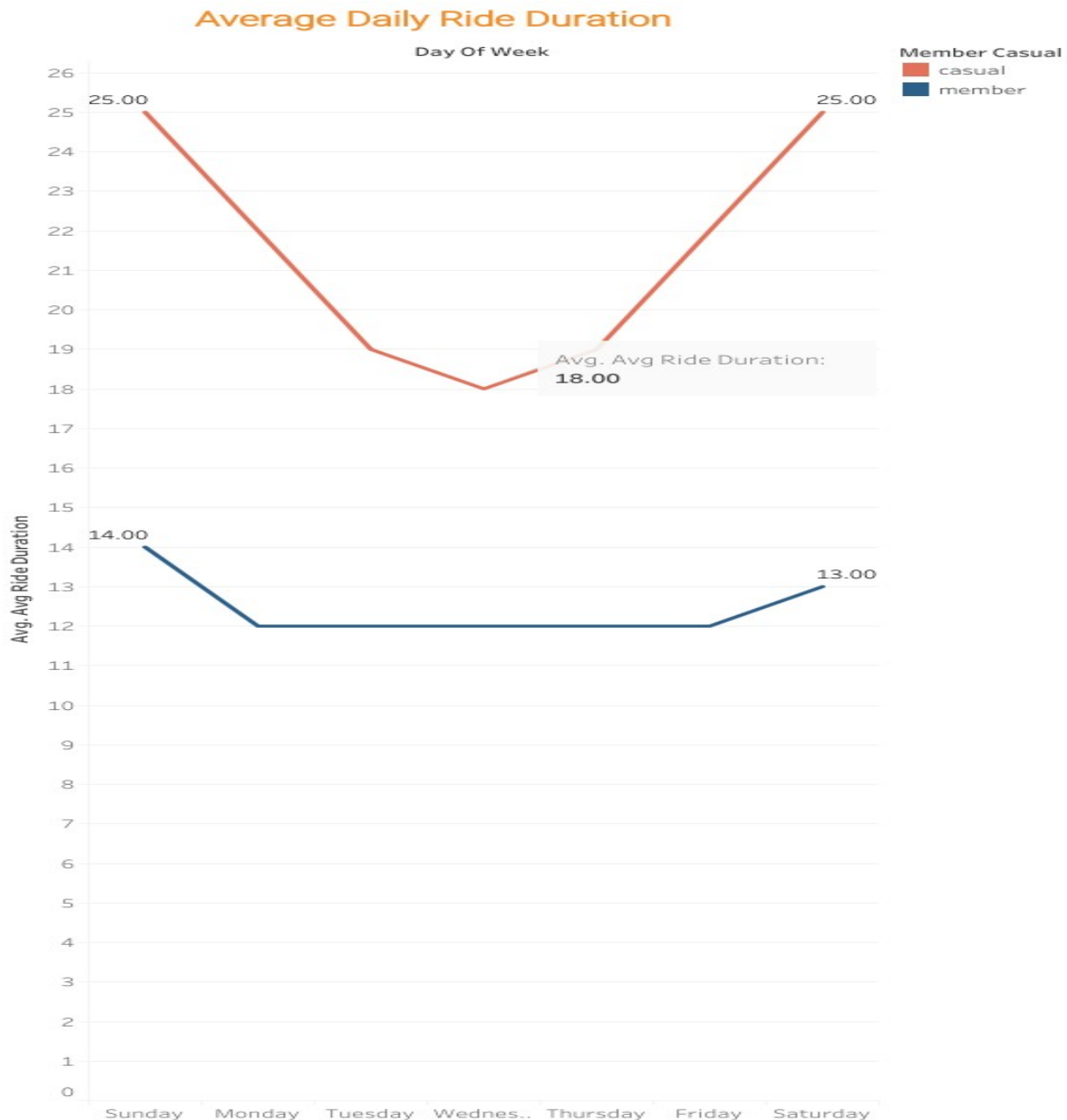




The chart above shows two peaks for member ridership during the day. One between 7 am to 8 am and the other during evening at around 4 pm to 6 pm. The number of trips by casual riders increases consistently during the day from 7 am and into the evening then decreases afterwards. The gap between casual riders and members is closest between 11 pm and 4 am due to a more significant drop in member ridership.

This observation shows that members may be using bikes for commuting to and from work during weekdays while casual riders use bikes more frequently during the day and on weekends for leisure purposes. Members and casual riders are mostly active in summer and spring.

5. **Ride Duration & Distance:** Trip duration compared to finding differences in the behavior of casual and member riders.



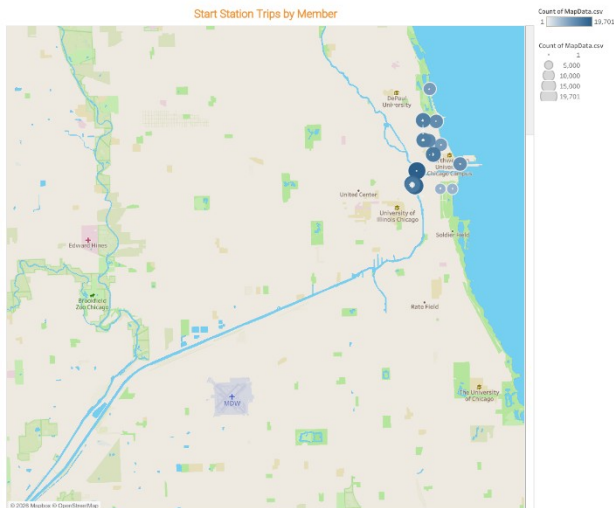
Analysis shows casual riders ride longer than members on average with peak duration occurring on weekends. But ride duration for members is consistent across all days.

**Note on chart design:** Most charts in this section highlight line-end values and min/max points rather than plotting every data point, to keep the visuals clean and focused on peaks, lows, and overall trend direction.

The conclusion from analysis indicates that casual commuters travel longer, twice the times as members but not as frequent as members. The casual riders take

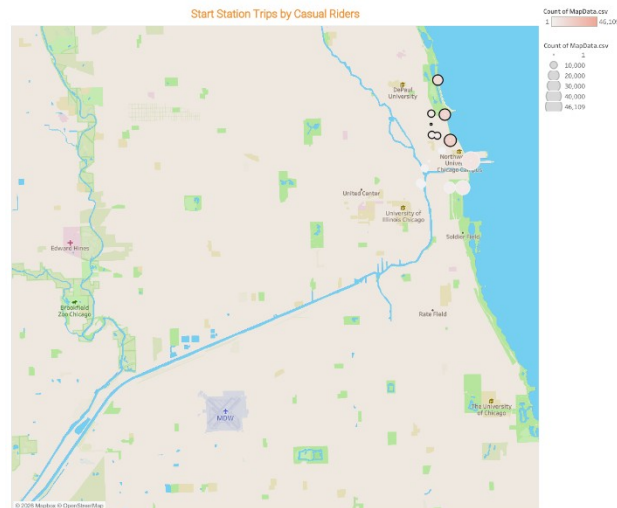
longer trips on weekends and during the day outside of commuting hours, during spring and summer which indicate their trips are for recreation.

## 6. Top Start Station Locations by Members (Apr 2025–Mar 2026)



Members' most frequent start stations cluster around universities, residential areas, hospitals, schools, banks, factories, train stations, and parks — consistent with commuting and everyday errands.

## 7. Top Start Station Locations by Casual Riders (Apr 2025–Mar 2026)



Casual riders' most frequent start stations cluster around museums, parks, beaches, and piers — consistent with recreational and sightseeing trips rather than daily commuting.

Together, these station patterns reinforce the time-of-day and duration findings above: members ride primarily for utility and account for the larger share of total trips (64.33%), while casual riders ride for leisure, take longer trips on average, and account for the remaining 35.67% of total trips.

## Recommendations

### 1. Targeted Advertising:

- o Use data-driven insights to create targeted ad campaigns that highlight the convenience and cost savings of annual membership.
- o Focus on digital channels like social media, email newsletters, and mobile apps to reach casual riders effectively.

### 2. Engaging Content:

- o Develop engaging content that demonstrates the benefits of being a member, such as ease of access, availability of different bike types, and exclusive member benefits.

- o Use testimonials and stories from current members to build trust and credibility.

### **3. Promotional Campaigns:**

- o Run limited-time promotions during peak season (August-September) and on weekends, when casual ridership is highest, to entice casual riders to upgrade to annual memberships.
- o Offer referral bonuses for existing members who bring in new annual members from the pool of casual riders.

### **4. Mobile App Enhancements:**

- o Improve the mobile app experience by adding features like ride tracking, personalized recommendations, and seamless membership upgrade options.
- o Push notifications about exclusive member benefits, upcoming promotions, and personalized offers based on casual riders' usage patterns.

## **Conclusion**

By understanding the distinct usage patterns of annual members and casual riders, Cyclistic can design effective marketing strategies to convert casual riders into annual members. Leveraging digital media, targeted advertising, engaging content, and well-timed promotions can maximize the impact and drive membership growth, ensuring future success for Cyclistic.